Programme: Computing and Digital Technologies

Student: Sofiya Serbina

Laboratory Work No. 2

REST API and Message Queues

Section A Implementation and Results

The revised Flask API provides synchronous /time and /echo requests and asynchronous text analysis. POST /jobs durably queues a message and returns HTTP 202 with a UUID. An independent worker reads the file queue and stores the result for GET /jobs/<job_id>.

```
curl -i http://127.0.0.1:5001/jobs/unknown
HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.9.6
Date: Fri, 30 Oct 2026 08:13:51 GMT
Content-Type: application/json
Content-Length: 51
Connection: close

{"server_time":"2026-10-30T08:13:51.612879+00:00"}
HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.9.6
Date: Fri, 30 Oct 2026 08:13:51 GMT
Content-Type: application/json
Content-Length: 26
Connection: close

{"message":"Hello queue"}
HTTP/1.1 400 BAD REQUEST
Server: Werkzeug/3.1.9 Python/3.9.6
Date: Fri, 30 Oct 2026 08:13:51 GMT
Content-Type: application/json
Content-Length: 69
Connection: close

{"error":"message must be a non-blank string of 1..4096 characters"}
HTTP/1.1 404 NOT FOUND
Server: Werkzeug/3.1.9 Python/3.9.6
Date: Fri, 30 Oct 2026 08:13:51 GMT
Content-Type: application/json
Content-Length: 26
Connection: close

{"error":"job not found"}
```

Figure 1. Terminal responses: /time and /echo return 200; invalid input returns 400; an unknown job returns 404.

```
(.venv) sofiyaserbina@MacBook-Pro-Sofiya Lab2_REST_Queues % JOB_ID="5d445131-c6c0-4d87-9cc8-ec022e2921a1"
curl -i "http://127.0.0.1:5001/jobs/$JOB_ID"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.9.6
Date: Fri, 30 Oct 2026 08:15:57 GMT
Content-Type: application/json
Content-Length: 180
Connection: close

{"created_at":"2026-10-30T08:14:55.136783+00:00","job_id":"5d445131-c6c0-4d87-9cc8-ec022e2921a1","payload":{"message":"Hello queue"},"status":"queued","task_type":"text_anal
ysis"}
```

Figure 2. Job 5d445131-c6c0-4d87-9cc8-ec022e2921a1 is queued with payload "Hello queue".

```
Press CTRL+C to quit
127.0.0.1 - - [30/Oct/2026 13:13:51] "GET /time HTTP/1.1" 200 -
127.0.0.1 - - [30/Oct/2026 13:13:51] "POST /echo HTTP/1.1" 200 -
127.0.0.1 - - [30/Oct/2026 13:13:51] "POST /echo HTTP/1.1" 400 -
127.0.0.1 - - [30/Oct/2026 13:13:51] "GET /jobs/unknown HTTP/1.1" 404 -
127.0.0.1 - - [30/Oct/2026 13:14:55] "POST /jobs HTTP/1.1" 202 -
127.0.0.1 - - [30/Oct/2026 13:15:57] "GET /jobs/5d445131-c6c0-4d87-9cc8-ec022e2921a1 HTTP/1.1" 200 -
127.0.0.1 - - [30/Oct/2026 13:17:19] "GET /jobs/ HTTP/1.1" 404 -
```

Figure 3. Flask records POST /jobs with status 202. This log is server evidence; it does not display the client response body.

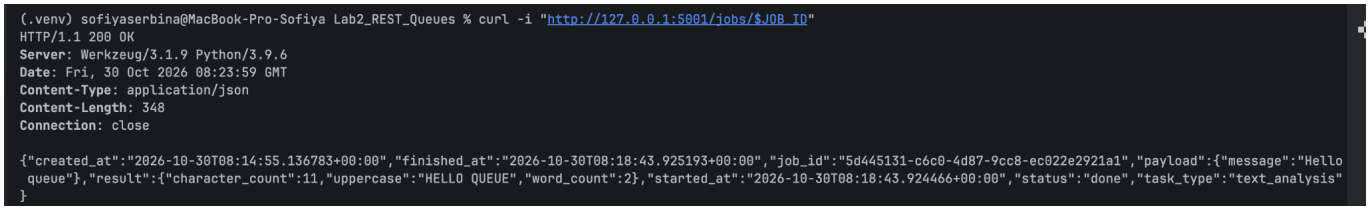


Figure 4. The same UUID is done: uppercase HELLO QUEUE, character_count 11, word_count 2, with lifecycle timestamps.

Section B Asynchronous Processing and Worker Recovery

HTTP 202 confirms acceptance, not completion. Jobs remain queued while the worker is stopped. The lifecycle is queued, processing, then done or failed. The worker claims a file, computes text statistics, saves the terminal result and only then removes the active queue record (ACK). Startup recovers incomplete claims; an existing saved result takes precedence over a leftover claim.

FIFO uses creation time and UUID for ties. Recovery provides at-least-once execution. Repeated text analysis is safe because it is deterministic and has no external side effects. A crash before ACK must not lose an already saved result.

Testing and Verification

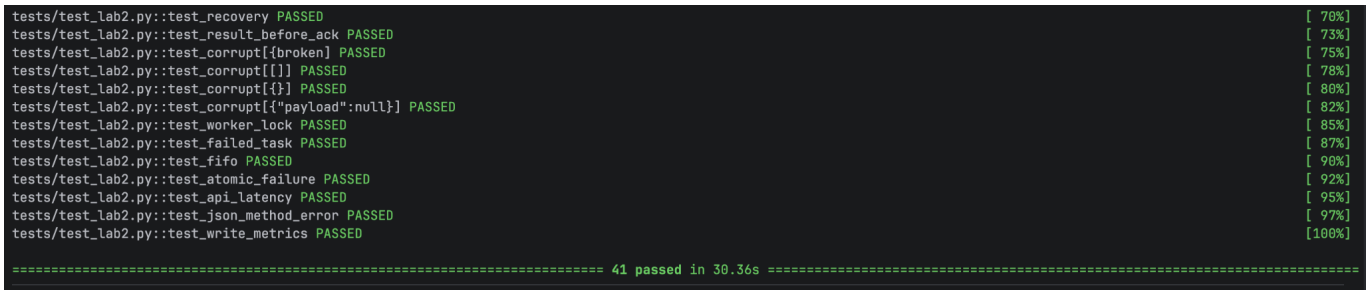


Figure 5. Student terminal run: 41 passed in 30.36s. The separate final rerun is recorded in evidence/pytest.txt.

The final rerun reported 41 passed in 30.34s, with 0 failed. Ten jobs persisted without a worker for 30.016 seconds; all ten resumed with correct results in 0.0228 seconds. The p95 of 100 test-client submissions was 0.606 ms. Tests also verified recovery after a killed process and preservation of the saved result after a failure before ACK.

Conclusion and Limitations

The tested implementation supports asynchronous processing and recovery on one local POSIX host with one worker. It does not provide authentication, distributed storage or a guarantee against every hardware failure. Original code and specification shared commit c5837d7. The honest remediation sequence was specification c4772f7, implementation 8652cab, then verification 7be01cd. OpenAI Codex assisted the remediation; the student reviews and explains the code.